thdocs

Release 0.1.0

Thai Pangsakulyanont

May 08, 2026

User Guide

Welcome to thdocs—a reusable documentation generator for Markdown projects.

1 Getting Started

Welcome to thdocs. This guide covers installation, basic project setup, and your first build.

1.1 Installation

Install thdocs from source using uv:

```
uv tool install -e . --reinstall
```

This installs the `thdocs` command globally.

1.2 Creating a New Project

To scaffold a new documentation project:

```
mkdir my-docs
cd my-docs
thdocs init
```

This creates:

- `thdocs.toml` — project metadata (title, author, version)
- `docs/index.md` — root page with a `{toctree}` directive
- `.gitignore` entry for the `_build/` output directory

1.3 Project Layout

A typical thdocs project looks like:

```
my-project/
  thdocs.toml
  docs/
    index.md
    getting-started.md
    cli.md
    adr/
      index.md
      0001-something.md
  _build/
    html/
      index.html
      ...
```

The source is pure Markdown (MyST format). Use `{toctree}` directives in `index.md` files to structure your navigation.

1.4 Authoring

Write Markdown files in `docs/` using MyST syntax:

```
# My Page

This is a paragraph with **bold** and *italic*.

[Link to another page](other-page.md)
```

Use Sphinx directives directly for structure:

````
# Contents

```{toctree}
subpage-1
subpage-2
```
````

See *CLI Reference* (page 5) for build and dev commands.

1.5 Next Steps

- Read the *CLI Reference* (page 5) to learn `thdocs build` and `thdocs dev`
- Check the *Kitchen Sink* (page 6) to see all supported Markdown constructs

2 Configuration

Project settings live in a `thdocs.toml` file at the project root. The file uses TOML[1] format.

[1] https://toml.io/en/

2.1 Structure

```toml
[site]
title = "My Project"
author = "Your Name"
version = "1.0.0"
```

2.2 Options

`[site].title`
: The project title. Used as the page `<title>` prefix, the site header, and the PDF document title.

  Required. If missing, `thdocs build` and `thdocs dev` will fail.

`[site].author`
: The author or organization name. Used in the PDF metadata, the Sphinx `author` field, and the site footer copyright notice.

  Optional. Defaults to an empty string. When set, the footer shows `© Copyright 2026, Author Name.`; when empty, it shows `© Copyright 2026.`

`[site].version`
: The project version string. Displayed in the site header and PDF metadata.

  Optional. Defaults to an empty string. Three forms are supported:

  1. **Plain string** — used as-is:

     ```toml
     version = "2.5.0"
     ```

  2. **JSON file path** — thdocs reads the file and extracts the `"version"` field:

     ```toml
     version = "package.json"
     ```

  3. **Auto-detect** — if `version` is omitted and `package.json` exists in the project root, the `"version"` field is read automatically.

2.3 Example

```toml
[site]
title = "Widget SDK"
author = "Acme Corp"
version = "2.5.0"
```

2.4 How It Works

`thdocs.toml` is the only configuration file you need. There is no `conf.py` to maintain — thdocs generates the Sphinx configuration internally based on this file's contents.

When you run `thdocs init`, a minimal `thdocs.toml` with just the `title` field is created automatically.

3 CLI Reference

thdocs provides three main commands: `build`, `dev`, and `init`.

3.1 thdocs init

Scaffolds a new documentation project in the current directory.

```
thdocs init
```

Creates:

- `thdocs.toml` with project metadata
- `docs/index.md` with a starter `{toctree}`
- Appends `_build/` to `.gitignore`

3.2 thdocs build

Builds the documentation site as static HTML (and optionally PDF).

```
thdocs build
```

Options:

- `--pdf` — also generate PDF output (via Sphinx LaTeX builder)
- `--strict` — treat warnings as errors (equivalent to `sphinx-build -W`)

Output:

- HTML: `_build/html/index.html`
- Search index: `_build/html/search/` (generated by Pagefind)

3.3 thdocs dev

Runs a live-reload development server.

```
thdocs dev
```

Options:

- `--host <addr>` — bind address (default: `0.0.0.0`)
- `--port <port>` — bind port (default: `20080`)

Example:

```
thdocs dev --host 127.0.0.1 --port 8888
```

Then open `http://127.0.0.1:8888/` in your browser. The page reloads automatically when you save changes to Markdown files.

Note: The Search tab shows “disabled in dev mode” since Pagefind only runs on full builds.

3.4 Configuration

See *Configuration* (page 3) for all available `thdocs.toml` options.

The Sphinx configuration file (`conf.py`) is generated internally—you do not need to create one.

4 Kitchen Sink

This page exercises every renderable construct supported by thdocs and MyST. Use this to visually verify that your theme renders all elements correctly.

4.1 Headings

Heading levels Headings h1–h4 are shown below (h1 is the page title).

Level 3 Heading

This is a Level 3 Heading for demonstration.

Level 4 Heading

This is a Level 4 Heading for demonstration.

4.2 Text Formatting

This paragraph demonstrates Text Formatting with **bold text**, *italic text*, and `inline code`. You can also combine them: ***bold italic***.

A link to example.com[2] and a *link to another page* (page 2) in the documentation.

4.3 Lists

Unordered Lists

This section shows Lists and Unordered Lists.

- Item one
- Item two
  - Nested item 2a
  - Nested item 2b
- Item three

Ordered Lists

This section shows Ordered Lists.

1. First step
2. Second step
   1. Sub-step 2.1
   2. Sub-step 2.2

[2] https://example.com

3. Third step

4.4 Blockquote

This section demonstrates a Blockquote.

> This is a blockquote. It stands out from the main text and is used to highlight important information, quotes, or tangential notes.
>
> Blockquotes can span multiple paragraphs.

4.5 Code Blocks

This section demonstrates Code Blocks.

A Python code block with syntax highlighting:

```python
#!/usr/bin/env python3
"""A simple example script."""

def greet(name: str) -> str:
    return f"Hello, {name}!"

if __name__ == "__main__":
    print(greet("World"))
```

A shell code block:

```bash
#!/bin/bash
echo "Building documentation..."
thdocs build --strict
```

4.6 Table

This section demonstrates a Table.

A table with header and content rows:

Feature	Supported	Notes
MyST Markdown	Yes	Full CommonMark + MyST extensions
Dark Theme	Yes	Custom thdocs theme with Tailwind
Search	Yes	Powered by Pagefind

4.7 Admonitions

This section demonstrates Admonitions.

Note

This section demonstrates a Note admonition.

Note

This is a note admonition. Use notes to provide additional context or helpful information.

Warning

This section demonstrates a Warning admonition.

Warning

This is a warning admonition. Use warnings to highlight important cautions or gotchas.

4.8 Keyboard Input

This section demonstrates Keyboard Input.

To run thdocs, press Ctrl + Enter or type:

```
thdocs dev
```

4.9 Details / Summary

This section demonstrates Details / Summary blocks.

This is hidden content that appears when you click the summary. It's useful for optional or advanced information that shouldn't clutter the main page.

You can include multiple paragraphs and other elements inside a details block.

4.10 Images and Figures

This section demonstrates Images and Figures.

An image with a caption using a figure element:

```
<figure>
  <img src="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='200'
↪height='100'%3E%3Crect fill='%23454443' width='200' height='100'/%3E%3Ctext x='50%25
↪' y='50%25' dominant-baseline='middle' text-anchor='middle' fill='%23e9e8e7'
↪font-family='Arimo' font-size='16'%3ESample Image%3C/text%3E%3C/svg%3E" alt="A
↪sample SVG image" width="200" height="100">
  <figcaption>This is a figure caption. It describes the image above.</figcaption>
</figure>
```

4.11 Horizontal Rule

This section demonstrates a Horizontal Rule.

A horizontal rule divides sections visually:

4.12 Superscript and Subscript

This section demonstrates Superscript and Subscript.

The chemical formula H2O uses subscript. E = mc2 uses superscript.

4.13 Thai Text

This section demonstrates Thai Text rendering.

นี่คือข้อความภาษาไทย — ทดสอบการแสดงผลฟอนต์ภาษาไทยใน thdocs ด้วย Sarabun ซึ่งรองรับภาษาไทยอย่างสมบูรณ์

4.14 Summary

This page demonstrates:

- ✓ Headings (h1–h4)
- ✓ Paragraph text with bold, italic, and inline code
- ✓ Links (external and cross-document)
- ✓ Ordered and unordered nested lists
- ✓ Blockquotes
- ✓ Fenced code blocks with syntax highlighting
- ✓ Tables with headers
- ✓ MyST admonitions (note, warning)
- ✓ HTML `<kbd>` elements
- ✓ HTML `<details><summary>` blocks
- ✓ Images with captions (via `<figure>`)
- ✓ Horizontal rules
- ✓ Superscript and subscript

All constructs should render cleanly with the thdocs dark theme.

5 Style Guide

This guide defines how to write documentation for thdocs projects. It is written for human authors and coding agents.

5.1 Philosophy

- **One idea per page.** Pages are navigation targets. If a page covers too much, split it.
- **Self-contained pages.** Any page might be the entry point (via search or index). It should make sense in isolation.
- **Flat hierarchy.** Use `{toctree}` to group pages by topic. Prefer two levels of depth; nest deeper only when it truly helps.

5.2 Page Structure

Every page starts with an h1 title. Use h2 for sections and h3 for subsections. Do not go deeper than h3. A short page may have no h2 at all if the content is focused enough.

```
# Page Title

Introductory paragraph that stands on its own.

## A Section

Content here.

### A Subsection

More content.
```

5.3 Navigation

Use {toctree} directives in section index.md files to define the page hierarchy. Keep :maxdepth: at 2.

```
```{toctree}
:caption: User Guide
:maxdepth: 2

getting-started
configuration
cli
```
```

Do not create orphan pages. Every page must be reachable from a {toctree}.

5.4 Index Entries

Use the inline {index} role to mark key concepts **at the point where they appear in the text**. The role anchors the index entry to that exact location, so the Index tab jumps the reader to the right paragraph.

Only index **nouns, commands, features, and domain concepts**. Do not index structural headings like "Summary" or "See Also".

```
## Installation

Install thdocs with {index}`uv`:

```bash
uv tool install thdocs
```
```

5.5 Configuration

Edit thdocs.toml to set the site title.

```

## Cross-References
```

(continues on next page)

(continued from previous page)

````
Prefer Markdown link syntax when you want custom link text:

```markdown
Please [configure](configuration.md) it before continuing.
````

Use the {doc} role when you want the target page title to appear as the link text:

```
For more information, see {doc}`configuration`.
```

Linking to Headings

Before referencing a heading from another page, give it an explicit target with (target)=:

```
(installation-steps)=
## Installation Steps

1. Run the installer.
```

Then reference it with {ref}:

```
See {ref}`installation-steps` for details.
```

Explicit targets are stable. Implicit heading anchors are available as a fallback, but prefer explicit ones.

5.6 Glossary

Create a glossary page only when the project has enough domain-specific terms to justify one.

````
```{glossary}
toctree
    A Sphinx directive that generates a table of contents tree.
```
````

Reference glossary terms with {term}:

```
The {term}`toctree` directive controls navigation.
```

5.7 Admonitions and Callouts

Use {note} for extra context and {warning} for cautions:

````
```{note}
This is supplementary information.
```

```{warning}
This is a caution.
```
````
````

Custom Titles

Use the {admonition} directive when you need a custom title:

```
```{admonition} Pro tip
:class: tip
You can customize the title of any admonition.
```
```

Collapsible Admonitions

Add :collapsible: to let readers expand or collapse the block. Use closed to hide the content by default, or open to show it:

```
```{admonition} Advanced configuration
:class: note
:collapsible: closed

This section is hidden until the reader expands it. Use collapsible admonitions for
↪optional or advanced details that should not clutter the main flow.
```
```

5.8 See Also

Use the {seealso} directive for all related reading sections:

```
```{seealso}
- {doc}`configuration`
- {doc}`cli`
```
```

5.9 Definition Lists

Use definition lists for options, parameters, or term explanations. The deflist extension is enabled.

```
term
: The definition of the term.

another-term
: Another definition.
```

5.10 Anti-Patterns

- Do not skip heading levels (e.g., h1 directly to h3).
- Do not write walls of text without structural breaks. Use headings, lists, and code blocks.
- Do not use bare Markdown links to internal pages when {doc} or explicit anchors are more robust.
- Do not index every heading. The index is a curated keyword list, not a mechanical dump.

6 Architecture Decision Records

This section documents key architectural decisions in thdocs.

6.1 thdocs is a CLI wrapper that exposes MyST/Sphinx authoring directly

Context. thdocs is a documentation generator the author wants to reuse across all their projects. The user-facing input is Markdown; PDF output and a custom theme are required.

Decision. thdocs is a thin CLI wrapper (`thdocs build|dev|init`) over Sphinx. Sphinx *configuration* (`conf.py`, extension wiring, theme paths, builder names) is hidden from users. Sphinx *content authoring* — MyST directives, notably `{toctree}`, plus `{index}`, `{glossary}`, `{seealso}`, `{ref}`, `{doc}` — is part of the user-facing contract. Authors write Markdown and use these directives directly.

Why. Sphinx is stable and mature; abstracting over its authoring directives would be premature complexity for no benefit. The custom theme is the part still in flux, which is what justifies a wrapper at all. Principle: hide config plumbing, embrace authoring vocabulary.

Rejected. (a) Treating Sphinx as fully invisible — would require reinventing toctree, cross-refs, the index, the glossary, and a dozen other directives, all of which already work. (b) Consuming Sphinx as a library with no wrapper — would force every project to own a `conf.py` and learn theme-loading mechanics, which contradicts the whole "Markdown in, site out" pitch.

6.2 thdocs's theme CSS is independent; @apiref/shell is a visual reference

Context. The thdocs site must share a design language with the author's other property, `dtinth/apiref`, whose `@apiref/shell` package owns the design tokens, prose treatments, and chrome styling. shell ships a compiled Tailwind v4 stylesheet plus four Lit web components (`<ar-shell>`, `<ar-header>`, `<ar-nav>`, `<ar-outline>`) consumed by apiref via CDN.

Decision. thdocs does not consume shell as a runtime dependency. shell is treated as a *visual reference*: tokens, prose treatments, and CSS idioms are copied into thdocs and adapted to fit Sphinx's HTML output shape. thdocs's theme has its own class names, its own Jinja templates, and its own compiled `thdocs.css` (Tailwind v4 + `@tailwindcss/typography`, compiled at theme-author time and shipped inside the Python wheel). End users do not need a Node toolchain.

Why. apiref's HTML is shaped by Lit components; Sphinx's HTML is shaped by Jinja templates and the toctree model. Adopting shell's classes wholesale would mean either reshaping Sphinx's output (fighting Sphinx) or shipping classes that don't match the markup. PDF output further rules out anything that requires Lit hydration. Independent CSS, with shell as inspiration, gives us alignment with both Sphinx and the LaTeX builder while preserving the visual family resemblance.

Rejected. (a) Linking shell's compiled `dist/styles.css` and emitting `.ar-*` classes from Sphinx templates — class names assume apiref's HTML shape, and the bundle doesn't include `prose` styles. (b) Embedding shell's Lit components and feeding them a JSON nav blob — breaks PDF, fights `toctree`, couples thdocs to a private API. (c) Extending shell to bundle a Sphinx-aware build — couples shell to thdocs's needs and defeats Tailwind's whole-program purging.

Consequence. Drift between the doc theme and the website is managed manually. Acceptable while shell's tokens are stable; if drift becomes painful we can elevate shared tokens to a real package later.

6.3 Sphinx is invoked with a `conf.py` shipped inside the thdocs package

Context. Sphinx requires a `confdir` containing a `conf.py`. thdocs needs to invoke Sphinx without exposing Sphinx config to users.

Decision. thdocs ships a static `conf.py` inside its own Python package (e.g. `src/thdocs/sphinx/conf.py`). The CLI invokes Sphinx with `-c <thdocs-pkg>/sphinx`, passing the user's project root via the `THDOCS_PROJECT` env var. The shipped `conf.py` reads `<project>/thdocs.toml` at config time and translates fields into Sphinx settings (project, author, extensions, theme path, static path).

Why. This is the most idiomatic Sphinx integration: `conf.py` is *designed* to be Python that reads its environment. It keeps the user's project free of Sphinx config files, and stays fully compatible with `sphinx-autobuild` (which `thdocs dev` relies on).

Rejected. (a) Generating `conf.py` per build into a temp dir — extra file thrash, more glue to maintain. (b) Using the programmatic `Sphinx(...)` API with `confoverrides` — loses `sphinx-autobuild` compatibility, since that tool wants `-c <dir>` semantics.

6.4 Contents sidebar uses Lit component with JSON tree data, not Sphinx's native toctree markup

Context. The Contents tab must render a full, interactive tree of all pages and sections (for CHM-style navigation) while keeping the initial HTML lean and avoiding duplication. Sphinx's `{toctree}` directive generates the tree as part of the HTML sidebar — nested `<ul>/<li>` with `<p class="caption">` section headers interspersed. This creates two problems: (a) embedding the full tree in every page's HTML bloats initial markup, especially in large docs with API references, and (b) the mixed HTML structure (captions as `<p>` elements, pages as `<li>` elements) requires two code paths in JavaScript to handle uniformly.

Decision. The build process generates a separate `toctree.json` file (via a Sphinx extension) containing the full tree structure. The sidebar HTML is minimal — just the current-page nav context, like normal Sphinx docs. A Lit-based `<tdhocs-tree>` web component fetches `toctree.json` at runtime and renders the full interactive tree. Tree state (expanded/collapsed sections, scroll position, focused link) is persisted in `sessionStorage`.

Why. This approach decouples data (the tree) from markup (the sidebar HTML), giving us:

- **Lean initial HTML.** Only the current page's nav context is embedded; the full tree is fetched on demand.
- **No duplication.** The tree is defined once (in `toctree.json`) and consumed by JavaScript, not duplicated in every page's HTML.
- **Unified structure.** Captions and pages become uniform tree nodes in JSON, with one JavaScript code path instead of two.
- **Full control.** A Lit component lets us implement exactly the UX we want (collapse/expand, scrolling, focus restoration, keyboard nav) without fighting Sphinx's rendered HTML.
- **Graceful degradation.** If JavaScript is unavailable, the minimal Sphinx nav is still present and functional.
- **Reusable component.** `<tdhocs-tree>` is a web component that could be reused or shared with other projects (e.g., apiref/shell).

Rejected. (a) Parse Sphinx's HTML tree in JavaScript — fragile if Sphinx's output changes, and the mixed `<p>` / `<li>` structure requires two code paths. (b) Embed the full tree in HTML and let JavaScript enhance it — bloats every page's initial HTML by duplicating the tree. (c) Use vanilla JavaScript instead of Lit — would work but loses the component model and reusability. (d) Fetch HTML instead of JSON — JSON is smaller and more structured; HTML would require parsing and reconstruction.

Consequence. The build process must generate `toctree.json` from Sphinx's internal toctree state. This requires a Sphinx extension hook (e.g., `config-inited` or `build-finished`). The tree JSON schema is owned by thdocs, not Sphinx, so the contract is explicit and stable.

6.5 Use Sphinx's built-in client-side search instead of Pagefind

Context. The sidebar has a Search tab; initially the plan was to run Pagefind as a post-build step over Sphinx's HTML output and use its JS API to power the tab. Pagefind required a separate binary (npm/PyPI), a post-build subprocess invocation, and its own indexing step.

Decision. thdocs uses Sphinx's built-in client-side search engine (`searchtools.js` + `searchindex.js` + `language_data.js`) instead of Pagefind. The search index (`searchindex.js`) is lazy-loaded when the Search tab is first activated. Results are rendered directly into the sidebar panel via the `Search` API — no full-page redirect.

Why. Sphinx already generates a complete, working client-side search engine as part of every build — a stemmed inverted index with full-text, title, index-entry, and object search. It is included with every Sphinx installation, needs no extra dependencies, requires no post-build step, and works in both production and dev mode (since `sphinx-autobuild` generates `searchindex.js` too). Pagefind would have added complexity (binary management, post-build hook, dev-mode special-casing) with no meaningful benefit for a documentation site of this scale.

Rejected. (a) Pagefind — adds a native binary dependency, a post-build step, and extra CI complexity for marginal improvement. (b) Full-page redirect to Sphinx's `search.html` — takes the user away from their current page, breaks the CHM-inspired sidebar-chrome pattern.

Consequence. The ~200 KB `searchindex.js` payload is loaded on demand when the Search tab is first activated. It is cached by the browser on subsequent page loads (it has a stable URL per build). The built-in `makeSearchSummary` fetches each result page to extract context snippets, which means extra HTTP requests — acceptable for the typical doc-site workload.

Architecture Decision Records (ADRs) capture the rationale behind significant design choices. Each ADR documents the context, decision, and consequences.

Index